ANALYZING EXONERATIONS USING LARGE LANGUAGE MODELS

Anshuman Singh[1], Xiangdi Lin[2], Shreya Balaji[1], Kyle Torres[3]
Project Advisors: Dr. Deanna Needell[2], Dr. Minxin Zhang[2]

[1]Department of Mathematics, Harvey Mudd College, Claremont, CA
[2]Department of Mathematics, University of California, Los Angeles, CA
[3]Department of Mathematics, Pomona College, Claremont, CA

Email: ansingh@g.hmc.edu, sbalaji@g.hmc.edu, linxd@g.ucla.edu, deanna@math.ucla.edu, minxinzhang@g.ucla.edu, kdta2021@mymail.pomona.edu

Abstract. The Innocence Center provides free legal services to people in prison in California. They receive numerous requests for consulting on case files, and processing and interpreting such large amounts of information poses a significant challenge for the Innocence Center. In this paper, we explore the use of large language models (LLMs) in analyzing wrongful incarceration cases using Non-Negative Matrix Factorizations (NMF) and their variants along with Support Vector Machines (SVM) to study underlying topics in murder case files. We also build our own extensions: Convex Semi-Supervised Non-Negative Matrix Factorizations (Convex SSNMF) and Kernel Semi-Supervised Non-Negative Matrix Factorizations (Kernel SSNMF) to help impose geometric constraints on our data and aid in classification and clustering. We ultimately aspire to make an interpretable and transparent recommender system for whether the provided case can be exonerated in appeal. The results explore the classification accuracies across various experiments where we anonymize names and demographic information to identify what features contribute to AI concluding exoneration.

1. Introduction

The California Innocence Center is an independent non-profit law firm dedicated to freeing the innocent from prison and assisting freed clients to reenter society amongst many others. The Innocence Center receives a large number of requests to process and interpret documents. This poses a significant strain on resources due to the challenge posed by interpreting and analyzing such large amounts of information to determine whether the client, in the Innocence Center's opinion, be exonerated or not.

Focusing on murder cases where data of appeal was close to date of conviction, we collected 69 cases of exonerated individuals in murder cases from the National Registry Of Exonerations [9], a repository of the Michigan State University College of Law, the Newkirk Center for Science and Society at the University of California—Irvine, and the University of Michigan Law School. We collected 69 cases of similar demographics as our exonerated individuals on state, county and year of appeal decision to ensure uniformity in case law being applied to prevent training bias on regional information.

Without access to a Legal Large Model, we generated summaries of cases using Open AI's GPT 4 tool based on evidence, testimony accounts, and whether it recommends the case be exonerated or not to identify implicit biases of the case summaries and how AI judges cases. The three categories of summaries were converted to embedding vectors using Open AI's "text-embedding-3-small" [10].

In this paper, we utilize Semi-Supervised Nonnegative Matrix Factorizations (SSNMF), Support Vector Machines (SVM), and our extensions: Convex Semi-Supervised Nonnegative Matrix Factorizations (Convex SSNMF) and Kernel Semi-Supervised Nonnegative Matrix Factorization (Kernel SSNMF). The goal is to understand what specific features the algorithm was using to classify whether the case should be exonerated and to increase the interpretability of LLMs in sensitive applications.

1.1. Contributions.

The contributions of this paper are as follows:

(1) We present a pipeline for converting open-source legal documents into text embeddings to support classification tasks related to exoneration.
(2) We introduce two novel algorithms—Convex Semi-Supervised Nonnegative Matrix Factorization (Convex SSNMF) and Kernel SSNMF—for analyzing and classifying high-dimensional data by reconstructing the feature matrix using SVM under convex geometric constraints.

(3) We establish convergence guarantees for the proposed algorithms.
(4) We evaluate our methods on publicly available datasets, including documents from exonerated and non-exonerated cases, demonstrating strong classification performance and interpretability via geometric constraints.
(5) We propose an embedding-aware anonymization strategy using GPT to de-bias input data while preserving classification and clustering performance.

1.2. **Organization.** In Section 2, we describe the dataset collection process, including demographic matching and our use of OpenAI embeddings. Section 3 reviews relevant literature on NMF-based models and legal-domain embeddings along with text inversion methods. Section 4 discusses our algorithmic extensions: Convex Semi-supervised Nonnegative Matrix Factorization (Convex-SSNMF) and Kernel Semi-supervised Nonnegative Matrix Factorization (Kernel SSNMF). We also provide proofs for its convergence and classification theory. Section 5 evaluates Convex SSNMF on multiple datasets, including UCI benchmark tasks and OpenAI-embedded legal case summaries. We compare performance against SSNMF and standard NMF, and assess classification accuracy across 10 randomized train-test splits using scikit-learn SVMs with both `linear` and `rbf` kernels under default hyperparameter settings. This baseline allows for a direct comparison without tuning advantages and supports our analysis of topics arising from convex geometric constraints. Lastly, we examine the role of demographic information by leveraging GPT to suggest names and locations it considers demographically neutral. These substitutions are applied prior to embedding and classification, allowing us to assess how the removal of demographic cues affects model performance.

2. Data

2.1. **Data Selection and Source Overview.** In this section, we outline our data selection process and briefly describe the methodology used to interpret the classification results. As noted earlier, we compiled a dataset of 69 male murder cases involving exonerated individuals from the National Registry of Exonerations, paired with an equal number of non-exonerated murder cases drawn from the same states. All court opinions were sourced from CaseText and Westlaw. To mitigate potential researcher bias, non-exonerated cases were randomly selected on a state-by-state basis. Two primary filters were applied: (1) all exonerations must have occurred within the past ten years, and (2) federal Supreme Court cases were excluded.

These restrictions served two main purposes. First, matching exonerated and non-exonerated cases by state minimized demographic and jurisdictional biases, ensuring greater homogeneity in legal language and court context. Second, restricting the dataset to recent cases reduced variability due to the introduction of new evidence in older trials, thereby improving the consistency of legal reasoning across the dataset.

Figures 1 and 2 illustrate the state-wise distribution of exonerated murder cases over the past ten years. Notably, 11 states had no entries post sampling; our analysis is therefore limited to jurisdictions with available data. Figures 3a and 3b highlight the demographic composition of our dataset, showing a high prevalence of Black male defendants. The sampled dataset in Figure 3b contains a higher proportion of white individuals compared to the original dataset. Additionally, Figure 4 shows that Black and Hispanic individuals waited the longest—by an average of 2.5 years—before being exonerated. Our sampling method inadvertently increased the average incarceration time prior to exoneration and reduced the recorded incarceration period for Hispanic individuals to approximately 11 years.

2.2. **Case Categorization and Embeddings Generation.** To convert text into effective embeddings for data analysis while abiding by document processing length constraints, we entered the following three prompts to get different summaries from GPT 4 for the case files:

- Prompt 1: *"Summarize evidence from this case."*
- Prompt 2: *"Evaluate how the accuracy and reliability of eyewitness testimony influenced the outcome of this case, considering factors such as the witnesses' credibility, consistency, and potential biases."*
- Prompt 3: *"Give me a good summary for this case to help the judge decide whether exonerated or non-exonerated."*

A generated sample summary is as follows (names anonymized for privacy):

> *The case involves two individuals, GKH and JDC, who were convicted of the 1992 murder of RSW based on circumstantial evidence. Both were sentenced to life imprisonment. The Innocence Project is representing the appellants in seeking the release of physical evidence,*

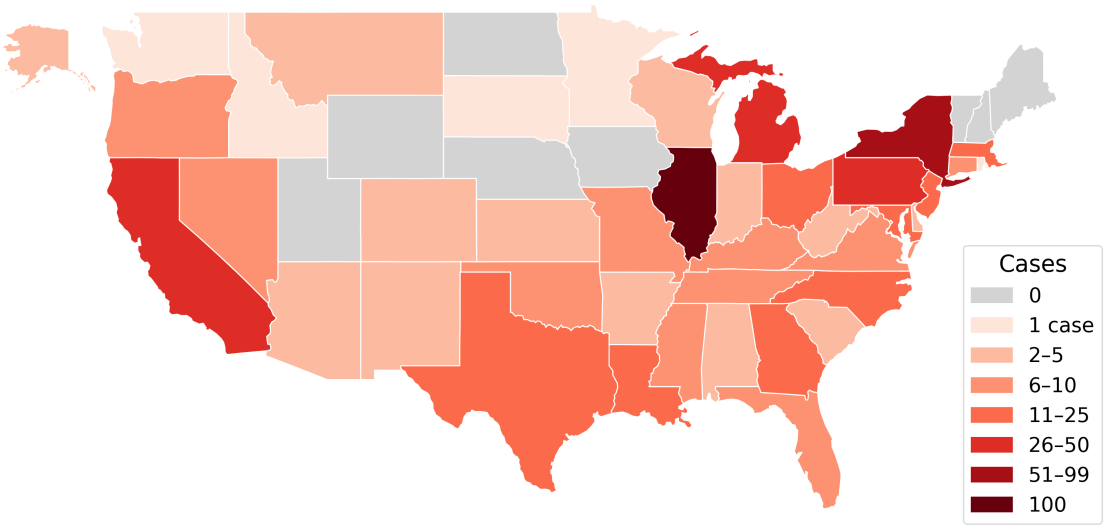


FIGURE 1. Geographic distribution of exoneration cases by state from 2014-2024. Data sourced from the *National Registry of Exonerations* [9]. States are not drawn to geographical scale.

> *specifically unidentified hairs found in the victim's hand, for DNA testing that was not available at the time of the trial. The court held that the appellants are entitled to the DNA testing they seek, as it may help prove their innocence.*

Upon reading the excerpts above, one might suspect that certain keywords, such as "Innocence Project," might directly contribute to higher classification accuracies. However, we put the case files through GPT to classify based on keywords only, and its classification accuracy hovered around 50%.

Therefore, we proceeded to more sophisticated methods, such as those that imposed geometric constraints. We hypothesized that Prompt 3 would yield the highest classification accuracy, as it explicitly asked for the verdict. At the same time, Prompt 2 would perform the worst, given that testimony is a subset of evidence. Not all cases contain substantive eyewitness testimony (This assertion about evidence performing worse than testimony was proved wrong as shown in Section 5.2). Due to the small size of the data set and summary length constraints, we hypothesized that in the best case, the overall classification accuracy would not be much higher than 50%, even for standard machine learning algorithms. Notably, our collaborating attorneys reported that they could not reliably determine whether a case should result in exoneration based on the short summaries alone. Since we could not access a Legal LLM, we employed OpenAI's 1536-dimensional text-embedding-3-small to encode the texts as vectors. text-embedding-3-small was trained on a broad Internet-scale corpus, and we, therefore, expected it to underperform on legal-specific tasks compared to a legal LLM's embeddings.

3. Mathematical Background

Notation: Here, $\mathbb{R}_+$ denotes non-negative real numbers, and we separate the positive and negative parts of a matrix A as given in Equations 1 and 2 [2].

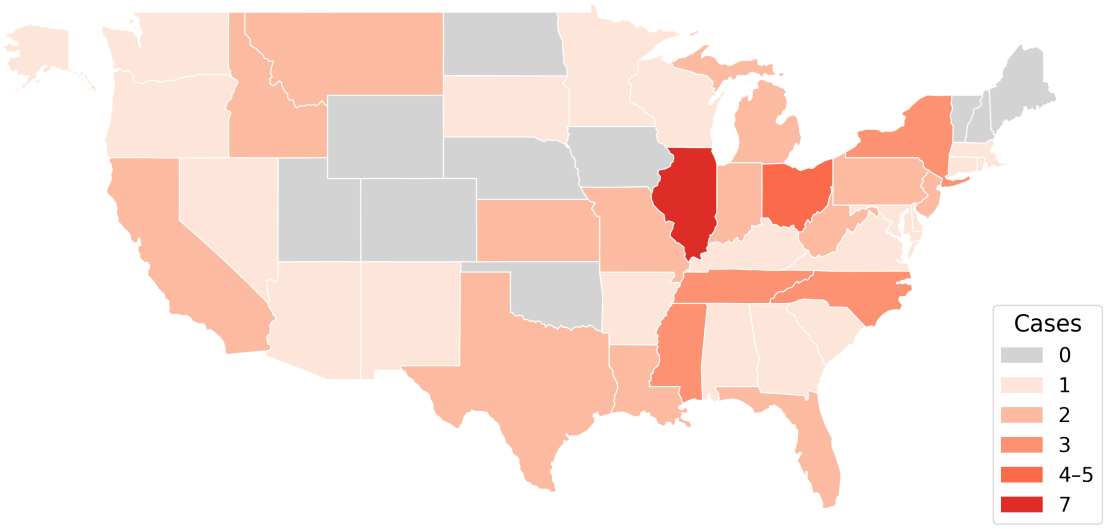
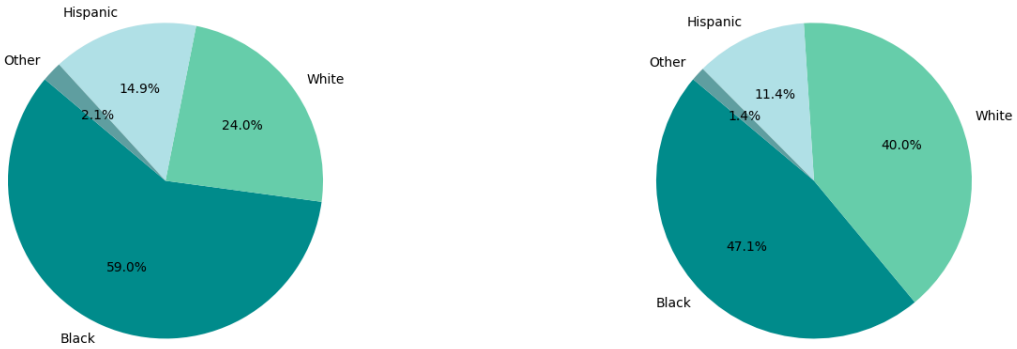


FIGURE 2. Geographic distribution of exoneration cases by state in sampled data from 2014-2024. States are not drawn to geographical scale.

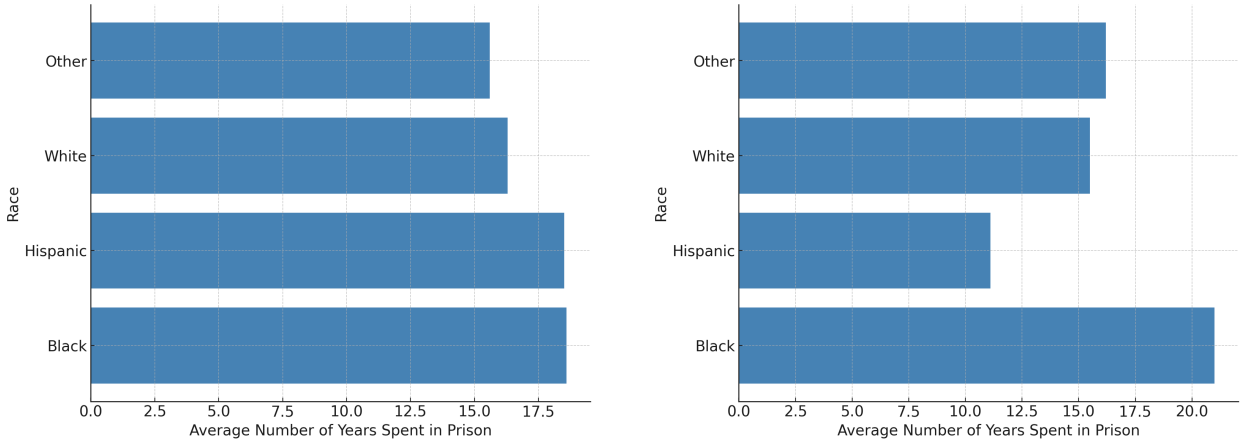


(A) Original Data (B) Sampled Data

FIGURE 3. Racial distribution in original vs. sampled dataset for exonerations.

$$\mathbf{A}_{ik}^{+} = \frac{|\mathbf{A}_{ik}| + \mathbf{A}_{ik}}{2}, \tag{1}$$

$$\mathbf{A}_{ik}^{-} = \frac{|\mathbf{A}_{ik}| - \mathbf{A}_{ik}}{2}. \tag{2}$$



(A) Original Data (B) Sampled Data

FIGURE 4. Comparison of average years spent in prison by race across original and sampled datasets for exonerated individuals.

3.1. **Non-negative matrix factorization (NMF) and extensions.** Non-negative matrix factorization (NMF) entails decomposing the data matrix, $\mathbf{X} \in {\mathbb{R}_+}^{m \times n}$ into two non-negative factor matrices: $A \in {\mathbb{R}_+}^{m \times k}$ and $S \in {\mathbb{R}_+}^{k \times n}$ to obtain a low-rank approximation of our original matrix. Hence, our factorization problem, assuming Gaussian noise maximum likelihood estimation, yields: $X \approx AS$ with objective function:

$$\min_{A \in \mathbb{R}_+^{m \times k},\ S \in \mathbb{R}_+^{k \times n}} \|X - AS\|_F^2 \tag{3}$$

The iterative updates and related proofs for the algorithms are provided in the works by Lee et al. [5] and Cho [1]. Non-negativity assists in model interpretability. For example, in images positive basis vectors indicate what parts add to give a composite whole unlike PCA where negativity entails components of images cancelling each other out and make interpreting contributing features more complicated [6]. We now discuss extensions of non-negative matrix factorization which are used to build our extensions in the next section.

3.1.1. *Semi Nonnegative Matrix Factorization (Semi-NMF).* [2]

Consider an unconstrained data matrix $X \in \mathbb{R}^{m \times n}$ which is decomposed into unconstrained basis matrix $F \in \mathbb{R}^{m \times k}$ and constrained ${G^T} \in {\mathbb{R}_+}^{k \times n}$, where F is the cluster centroids matrix and G is the cluster indicators matrix. The pseudo-code is provided in Algorithm 1.

Algorithm 1 Semi Nonnegative Matrix Factorization (Semi-NMF)

procedure SEMINMF($\mathbf{X}$, numClusters, N, tolerance)
  Initialize $\mathbf{G}^{(0)}$ = indicator matrix from K-Means($\mathbf{X}$, numClusters)
  Compute $\mathbf{F}^{(0)} = \mathbf{X}\mathbf{G}^{(0)} \left((\mathbf{G}^{(0)})^\top \mathbf{G}^{(0)} \right)^{-1}$
  for $k = 1, \ldots, N$ **do**
    $\mathbf{F}^{(k)} = \mathbf{X}\mathbf{G}^{(k-1)} \left((\mathbf{G}^{(k-1)})^\top \mathbf{G}^{(k-1)} \right)^{-1}$
    for each $i = 1, \ldots, n$ and $k = 1, \ldots, r$ **do**
      $$G_{ik}^{(t)} = G_{ik}^{(t-1)} \cdot \sqrt{\frac{\left[(\mathbf{X}^\top \mathbf{F}^{(t)})^+\right]_{ik} + \left[\mathbf{G}^{(t-1)} (\mathbf{F}^{(t)\top} \mathbf{F}^{(t)})^-\right]_{ik}}{\left[(\mathbf{X}^\top \mathbf{F}^{(t)})^-\right]_{ik} + \left[\mathbf{G}^{(t-1)} (\mathbf{F}^{(t)\top} \mathbf{F}^{(t)})^+\right]_{ik}}}$$
    reconstruction = $\|\mathbf{X} - \mathbf{F}^{(k)} (\mathbf{G}^{(k)})^\top\|_F$
    if reconstruction ≤ tolerance **then**
      break
  return $\mathbf{F}^{(k)}, \mathbf{G}^{(k)}$

3.1.2. *Convex Nonnegative Matrix Factorization (Convex-NMF).* [2]

Consider an unconstrained data matrix $X \in \mathbb{R}^{m\times n}$. We decompose X into a constrained basis matrices $F \in {\mathbb{R}_+}^{m\times k}$ and $G^T \in {\mathbb{R}_+}^{k\times n}$. The column space of F is formed as convex combinations of X as follows: $\mathbf{f_l} = w_{1l}\mathbf{x_1} + \cdots + w_{nl}\mathbf{x_n}$, yielding $F = XW$. The pseudocode for the algorithm is provided in Algorithm 2 with a similar objective function to Equation 3.

Convexity provides better clustering and model interpretability since each latent feature is directly tied to the original data vectors because they are bound in the convex hull of the original data.

Algorithm 2 Convex Nonnegative Matrix Factorization (Convex-NMF)

procedure ConvexNMF($\mathbf{X}$, numClusters, N, tolerance, method)
  if method = "kmeans" **then**
    Initialize $\mathbf{G}^{(0)}$ = indicator matrix from K-Means($\mathbf{X}$, numClusters)
    $\mathbf{W}^{(0)} = \mathbf{G}^{(0)}\left((\mathbf{G}^{(0)})^\top \mathbf{G}^{(0)}\right)^{-1}$
  else if method = "nmf" **then**
    Use existing NMF or Semi-NMF solution $\mathbf{G}$
    $\mathbf{W} = \mathbf{G}\left(\mathbf{G}^\top \mathbf{G}\right)^{-1}$
    $\mathbf{W}^{(0)} = \mathbf{W}^+ + 0.2 \cdot \langle \mathbf{W}^+ \rangle$
    $\mathbf{G}^{(0)} = \mathbf{G} + 0.2$
  for $t = 1, \ldots, N$ **do**
    for each $i = 1, \ldots, n$ and $j = 1, \ldots, k$ **do**
      $G_{ij}^{(t)} = G_{ij}^{(t-1)} \cdot \sqrt{\frac{\left[(\mathbf{X}^\top\mathbf{X})^+\mathbf{W}^{(t-1)}\right]_{ij} + \left[\mathbf{G}^{(t-1)}\mathbf{W}^{(t-1)\top}(\mathbf{X}^\top\mathbf{X})^-\mathbf{W}^{(t-1)}\right]_{ij}}{\left[(\mathbf{X}^\top\mathbf{X})^-\mathbf{W}^{(t-1)}\right]_{ij} + \left[\mathbf{G}^{(t-1)}\mathbf{W}^{(t-1)\top}(\mathbf{X}^\top\mathbf{X})^+\mathbf{W}^{(t-1)}\right]_{ij}}}$
    for each $i = 1, \ldots, n$ and $j = 1, \ldots, k$ **do**
      $W_{ij}^{(t)} = W_{ij}^{(t-1)} \cdot \sqrt{\frac{\left[(\mathbf{X}^\top\mathbf{X})^+\mathbf{G}^{(t)}\right]_{ij} + \left[(\mathbf{X}^\top\mathbf{X})^-\mathbf{W}^{(t-1)}\mathbf{G}^{(t)\top}\mathbf{G}^{(t)}\right]_{ij}}{\left[(\mathbf{X}^\top\mathbf{X})^-\mathbf{G}^{(t)}\right]_{ij} + \left[(\mathbf{X}^\top\mathbf{X})^+\mathbf{W}^{(t-1)}\mathbf{G}^{(t)\top}\mathbf{G}^{(t)}\right]_{ij}}}$
    reconstruction = $\left\|\mathbf{X} - \mathbf{X}\mathbf{W}^{(t)}\mathbf{G}^{(t)\top}\right\|_F$
    if reconstruction $\le$ tolerance **then**
      break
  return $\mathbf{W}^{(t)}, \mathbf{G}^{(t)}$

3.1.3. *Semi-Supervised Nonnegative Matrix Factorization (SSNMF).* [7]

SSNMF factorizes a data matrix $X \in \mathbb{R}_+^{m\times n}$ (where m is the number of terms and n is the number of samples) and a label matrix $Y \in \mathbb{R}_+^{k\times n}$ (where k is the number of classes and each column of Y is a binary vector) with the objective function in Equation 4.

$$\min_{\mathbf{A}\in\mathbb{R}_+^{\mathbf{m}\times\mathbf{r}},\mathbf{B}\in\mathbb{R}_+^{k\times r},\mathbf{S}\in\mathbb{R}_+^{r\times n}} \left\|\mathbf{W}\odot(\mathbf{X}-\mathbf{AS})\right\|_F^2 + \lambda\left\|\mathbf{L}\odot(\mathbf{Y}-\mathbf{BS})\right\|_F^2 \tag{4}$$

where λ is a weight parameter.

Here, $\mathbf{W} \in \mathbb{R}_+^{m\times n}$ is a binary weight matrix such that

$$\mathbf{W}_{ij} = \begin{cases} 1, & \text{if } \mathbf{X}_{ij} \text{ is observed} \\ 0, & \text{otherwise} \end{cases} \tag{5}$$

$\mathbf{A} \in \mathbb{R}_+^{m\times r}$ is the basis matrix for $\mathbf{X}$ where r is the number of classes, and $\mathbf{S} \in \mathbb{R}_+^{r\times n}$ is the feature matrix or the joint-factorization matrix. $\mathbf{L}$ is a weight matrix which handles missing entries and is 1 if the label is know and 0 otherwise. $\mathbf{B}$ is a basis matrix for the label matrix $\mathbf{Y}$ such that $\mathbf{B} \in \mathbb{R}_+^{k\times r}$. The pseudocode is provided in Algorithm 3.

Before classification, we reconstruct the feature matrix $\mathbf{S}_{\text{test}}$ by computing $\mathbf{S}_{\text{test}} = \mathbf{A}^\dagger\mathbf{S}_{\text{train}}$, where $\mathbf{S}_{\text{train}}$ is obtained by running the algorithm on the training data set. Following reconstruction, we perform Support Vector Machine (SVM) classification.

Algorithm 3 Semi-Supervised Nonnegative Matrix Factorization (SSNMF)

procedure SSNMF($\mathbf{X}, \mathbf{Y}, \mathbf{W}, \mathbf{L}, \lambda, r, N,$ tolerance)
  Initialize $\mathbf{A}^{(0)} \in \mathbb{R}_+^{m\times r}$, $\mathbf{B}^{(0)} \in \mathbb{R}_+^{k\times r}$, $\mathbf{S}^{(0)} \in \mathbb{R}_+^{r\times n}$
  for $t = 1, \ldots, N$ **do**
    $\mathbf{A}^{(t)} = \mathbf{A}^{(t-1)} \odot \frac{[\mathbf{W}\odot\mathbf{X}]\mathbf{S}^{(t-1)T}}{[\mathbf{W}\odot(\mathbf{A}^{(t-1)}\mathbf{S}^{(t-1)})]\mathbf{S}^{(t-1)T}}$
    $\mathbf{B}^{(t)} = \mathbf{B}^{(t-1)} \odot \frac{[\mathbf{L}\odot\mathbf{Y}]\mathbf{S}^{(t-1)T}}{[\mathbf{L}\odot(\mathbf{B}^{(t-1)}\mathbf{S}^{(t-1)})]\mathbf{S}^{(t-1)T}}$
    $\mathbf{S}^{(t)} = \mathbf{S}^{(t-1)} \odot \frac{\mathbf{A}^{(t)T}[\mathbf{W}\odot\mathbf{X}]+\lambda\mathbf{B}^{(t)T}[\mathbf{L}\odot\mathbf{Y}]}{\mathbf{A}^{(t)T}[\mathbf{W}\odot(\mathbf{A}^{(t)}\mathbf{S}^{(t-1)})]+\lambda\mathbf{B}^{(t)T}[\mathbf{L}\odot(\mathbf{B}^{(t)}\mathbf{S}^{(t-1)})]}$
    reconstruction = $\left\|\mathbf{W}\odot(\mathbf{X}-\mathbf{A}^{(t)}\mathbf{S}^{(t)})\right\|_F^2 + \lambda\left\|\mathbf{L}\odot(\mathbf{Y}-\mathbf{B}^{(t)}\mathbf{S}^{(t)})\right\|_F^2$
    if reconstruction ≤ tolerance **then**
      break
  return $\mathbf{A}^{(t)}, \mathbf{B}^{(t)}, \mathbf{S}^{(t)}$

3.1.4. *Kernel Nonnegative Matrix Factorization (Kernel-NMF).* Kernel-NMF entails taking the data matrix by mapping the column vectors (data vectors) into a different space as follows:

$$\mathbf{x}_i \to \phi(\mathbf{x}_i), \quad \text{for } i = 1, 2, \ldots, n$$

and then updating it using the rules of Convex-NMF as shown in Algorithm 4.

Algorithm 4 Kernel Nonnegative Matrix Factorization (Kernel-NMF)

procedure KernelNMF($\mathbf{X}$, numClusters, N, tolerance, kernel, method)
  Compute $\mathbf{K} = \phi(\mathbf{X})^T\phi(\mathbf{X})$
  Apply ConvexNMF to $\mathbf{K}$ using Algorithm 2
  return $\mathbf{W}, \mathbf{G}$

3.2. **Inversions of Embedding Vectors to Text.** Morris et al. [8] explore the task of reconstructing text from embeddings generated by models such as OpenAI's "text-embedding-3-small". Given a token sequence $x \in \mathbb{V}^n$, an encoder maps it to a fixed-length embedding vector $e \in \mathbb{R}^d$. The goal is to recover x such that its embedding closely matches e, which can be formalized as:

$$\hat{x} = \arg\max_x \cos(\phi(x), e)$$

where $\phi(x)$ denotes the embedding function.

They introduce `Vec2Text`, an iterative approach that refines an initial text hypothesis to better align with the target embedding. At iteration t, the probability distribution over possible texts is updated as:

$$p(x^{(t+1)}|e) = \sigma_{x^{(t)}}\, p(x^{(t)}|e)\, p(x^{(t)}|e, x^{(t)}, \hat{e}^{(t)})$$

where $\hat{e}^{(t)} = \phi(x^{(t)})$, and $\sigma_{x^{(t)}}$ denotes a contextual adjustment mechanism.

The initial guess $x^{(0)}$ is typically generated by a learned inversion model:

$$p(x^{(0)}|e) = p(x^{(t+1)}|e, \phi, \phi, \phi(\phi))$$

Embedding-to-Sequence Transformation. To reconstruct the sequence, the method defines a transformation:

$$\text{EmbToSeq}(e) = W_2\sigma(W_1 e)$$

where $W_1 \in \mathbb{R}^{h\times d}$, $W_2 \in \mathbb{R}^{sd_{\text{enc}}\times h}$, and σ is a non-linear activation function.

The network input includes concatenated representations:

$$\text{concat}\left(\text{EmbToSeq}(e), \text{EmbToSeq}(\hat{e}^{(t)}), \text{EmbToSeq}(e - \hat{e}^{(t)}), (w_1, \ldots, w_n)\right)$$

Implementation. The `Vec2Text` package reconstructs text from embeddings using transformer-based models. It supports inversion of embeddings from popular encoders, enabling both qualitative and quantitative evaluation of recovered text. Even with poor initialization, it achieves a high BLEU score. The results suggest that while perfect inversion is not achievable, text reconstructions can still reveal significant information about the underlying embedding structure and semantic meaning. However, such techniques are not applicable to topic matrices and do not provide semantic insight into how algorithm's classify embeddings post distortions due to matrix operations.

4. New Algorithmic Extensions and Theory

In the present section, we combine Section 3.1.2 and Section 3.1.3 to analyze our two major contributions: Convex SSNMF and Kernel SSNMF.

4.1. **Algorithmic Extensions.**

4.1.1. *Convex Semi-Supervised Nonnegative Matrix Factorization (Convex-SSNMF).* Consider a data matrix $\mathbf{X} \in \mathbb{R}_+^{m\times n}$ (where m is the number of terms and n is the number of samples) and a label matrix $\mathbf{Y} \in \mathbb{R}_+^{k\times n}$ (where k is the number of classes and each column of $\mathbf{Y}$ is a binary vector) with the following factorization problem:

$$\begin{bmatrix} \mathbf{X} \\ \sqrt{\lambda}\mathbf{Y} \end{bmatrix} = \begin{bmatrix} \mathbf{X} \\ \sqrt{\lambda}\mathbf{Y} \end{bmatrix} \mathbf{W}\mathbf{G}^\top \tag{6}$$

subject to the following objective function:

$$\min_{\mathbf{W}\in\mathbb{R}_+^{n\times k},\, \mathbf{G}\in\mathbb{R}_+^{k\times n}} \left\| \begin{bmatrix} \mathbf{X} \\ \sqrt{\lambda}\mathbf{Y} \end{bmatrix} - \begin{bmatrix} \mathbf{X} \\ \sqrt{\lambda}\mathbf{Y} \end{bmatrix} \mathbf{W}\mathbf{G}^\top \right\|_F^2 \tag{7}$$

where

$$\sum_{i=1}^{n} W_{ij} = 1 \quad \text{for each } j = 1, \ldots, k \tag{8}$$

We note that by rewriting $A = \mathbf{X}\mathbf{W}$, $B = \sqrt{\lambda}\mathbf{Y}\mathbf{W}$, and $S = \mathbf{G}^\top$, the optimization problem (7) becomes equivalent to the optimization problem (4). The iterative updates for the algorithm are similar to those provided for Convex NMF in [2]. The pseudocode is provided in Algorithm 5.

4.1.2. *Kernel Semi-Supervised Nonnegative Matrix Factorization (Kernel SSNMF).* Consider a data matrix $\mathbf{X} \in \mathbb{R}_+^{m\times n}$ (where m is the number of terms and n is the number of samples) and a label matrix $\mathbf{Y} \in \mathbb{R}_+^{k\times n}$ (where k is the number of classes and each column of $\mathbf{Y}$ is a binary vector). Similiar to Algorithm 4, we map the each column vector of X to a higher dimensional space to obtain $\phi(\mathbf{X})$. Thus, our factorization problem becomes:

$$\begin{bmatrix} \phi(\mathbf{X}) \\ \sqrt{\lambda}\mathbf{Y} \end{bmatrix} = \begin{bmatrix} \phi(\mathbf{X}) \\ \sqrt{\lambda}\mathbf{Y} \end{bmatrix} \mathbf{W}\mathbf{G}^\top \tag{9}$$

subject to the following objective function:

$$\min_{\mathbf{W}\in\mathbb{R}_+^{n\times k},\, \mathbf{G}\in\mathbb{R}_+^{k\times n}} \left\| \begin{bmatrix} \phi(\mathbf{X}) \\ \sqrt{\lambda}\mathbf{Y} \end{bmatrix} - \begin{bmatrix} \phi(\mathbf{X}) \\ \sqrt{\lambda}\mathbf{Y} \end{bmatrix} \mathbf{W}\mathbf{G}^\top \right\|_F^2 \tag{10}$$

where

$$\sum_{i=1}^{n} W_{ij} = 1 \quad \text{for each } j = 1, \ldots, k$$

We note that by rewriting $A = \phi(\mathbf{X})\mathbf{W}$, $B = \sqrt{\lambda}\mathbf{Y}\mathbf{W}$, and $S = \mathbf{G}^\top$, the optimization problem (10) becomes equivalent to the optimization problem (4). The iterative updates follow those of Convex NMF as proposed in [2]. The full algorithm is provided in Algorithm 6 with an alternative way to calculate the reconstruction

Algorithm 5 Convex Semi-Supervised Nonnegative Matrix Factorization (Convex SSNMF)

procedure ConvexSSNMF($\mathbf{X}, \mathbf{Y}, \lambda, \text{numClusters}, N, \text{tolerance}, \text{method}$)
  Stack: $\mathbf{Z} = \begin{bmatrix} \mathbf{X} \\ \sqrt{\lambda}\mathbf{Y} \end{bmatrix}$
  if method = "kmeans" **then**
    Initialize $\mathbf{G}^{(0)}$ = indicator matrix from K-Means($\mathbf{Z}, \text{numClusters}$)
    $\mathbf{W}^{(0)} = \mathbf{G}^{(0)} \left((\mathbf{G}^{(0)})^T \mathbf{G}^{(0)} \right)^{-1}$
  else if method = "semi_nmf" **then**
    Use existing $\mathbf{G}$ from Semi-NMF on $\mathbf{Z}$
    $\mathbf{W} = \mathbf{G} \left(\mathbf{G}^T \mathbf{G} \right)^{-1}$
    $\mathbf{W}^{(0)} = \mathbf{W}^+ + 0.2 \cdot \langle \mathbf{W}^+ \rangle$
    $\mathbf{G}^{(0)} = \mathbf{G} + 0.2$
  else if method = "random" **then**
    Initialize $\mathbf{G}^{(0)}$ with random nonnegative values
    $\mathbf{W}^{(0)} = \mathbf{G}^{(0)} \left(\mathbf{G}^{(0)T} \mathbf{G}^{(0)} \right)^{-1}$
  for $t = 1, \ldots, N$ **do**
    $\mathbf{G}^{(t)} = \mathbf{G}^{(t-1)} \cdot \sqrt{\frac{[(\mathbf{Z}^T\mathbf{Z})^+\mathbf{W}^{(t-1)}] + [\mathbf{G}^{(t-1)}\mathbf{W}^{(t-1)T}(\mathbf{Z}^T\mathbf{Z})^-\mathbf{W}^{(t-1)}]}{[(\mathbf{Z}^T\mathbf{Z})^-\mathbf{W}^{(t-1)}] + [\mathbf{G}^{(t-1)}\mathbf{W}^{(t-1)T}(\mathbf{Z}^T\mathbf{Z})^+\mathbf{W}^{(t-1)}]}}$
    $\mathbf{W}^{(t)} = \mathbf{W}^{(t-1)} \cdot \sqrt{\frac{[(\mathbf{Z}^T\mathbf{Z})^+\mathbf{G}^{(t)}] + [(\mathbf{Z}^T\mathbf{Z})^-\mathbf{W}^{(t-1)}\mathbf{G}^{(t)T}\mathbf{G}^{(t)}]}{[(\mathbf{Z}^T\mathbf{Z})^-\mathbf{G}^{(t)}] + [(\mathbf{Z}^T\mathbf{Z})^+\mathbf{W}^{(t-1)}\mathbf{G}^{(t)T}\mathbf{G}^{(t)}]}}$
    reconstruction = $\|\mathbf{Z} - \mathbf{Z}\mathbf{W}^{(t)}\mathbf{G}^{(t)T}\|_F$
    if reconstruction ≤ tolerance **then**
      break
  return $\mathbf{W}^{(t)}, \mathbf{G}^{(t)}$

error rather than computing $\phi(\mathbf{X})$ explicitly. It should also be noted that for a linear kernel, Kernel SSNMF becomes equivalent to Convex SSNMF, since the kernel matrix satisfies $\phi(\mathbf{X})^\top \phi(\mathbf{X}) = \mathbf{X}^\top \mathbf{X}$ in this case.

Algorithm 6 Kernel Semi-Supervised Nonnegative Matrix Factorization (Kernel SSNMF)

procedure KernelSSNMF($\mathbf{X}, \mathbf{Y}, \lambda, \text{numClusters}, N, \text{tolerance}, \text{method}$)
  Compute $\mathbf{K} = \phi(\mathbf{X})^\top \phi(\mathbf{X})$
  Compute $\mathbf{D} = \mathbf{K} + \lambda \cdot \mathbf{Y}^T\mathbf{Y}$
  if method = "kmeans" **then**
    Initialize $\mathbf{G}^{(0)}$ = indicator matrix from K-Means($\mathbf{D}, \text{numClusters}$)
    $\mathbf{W}^{(0)} = \mathbf{G}^{(0)} \left((\mathbf{G}^{(0)})^T \mathbf{G}^{(0)} \right)^{-1}$
  else method = "random"
    Initialize $\mathbf{G}^{(0)}$ with random nonnegative values
    $\mathbf{W}^{(0)} = \mathbf{G}^{(0)} \left(\mathbf{G}^{(0)T} \mathbf{G}^{(0)} \right)^{-1}$
  for $t = 1, \ldots, N$ **do**
    $\mathbf{G}^{(t)} = \mathbf{G}^{(t-1)} \cdot \sqrt{\frac{[\mathbf{D}^+\mathbf{W}^{(t-1)}] + [\mathbf{G}^{(t-1)}\mathbf{W}^{(t-1)T}\mathbf{D}^-\mathbf{W}^{(t-1)}]}{[\mathbf{D}^-\mathbf{W}^{(t-1)}] + [\mathbf{G}^{(t-1)}\mathbf{W}^{(t-1)T}\mathbf{D}^+\mathbf{W}^{(t-1)}]}}$
    $\mathbf{W}^{(t)} = \mathbf{W}^{(t-1)} \cdot \sqrt{\frac{[\mathbf{D}^+\mathbf{G}^{(t)}] + [\mathbf{D}^-\mathbf{W}^{(t-1)}\mathbf{G}^{(t)T}\mathbf{G}^{(t)}]}{[\mathbf{D}^-\mathbf{G}^{(t)}] + [\mathbf{D}^+\mathbf{W}^{(t-1)}\mathbf{G}^{(t)T}\mathbf{G}^{(t)}]}}$
    reconstruction = $\text{Tr}(\mathbf{D} - 2\mathbf{D}\mathbf{W}^{(t)}\mathbf{G}^{(t)T} + \mathbf{G}^{(t)}\mathbf{W}^{(t)T}\mathbf{D}\mathbf{W}^{(t)}\mathbf{G}^{(t)T})$
    if reconstruction ≤ tolerance **then**
      break
  return $\mathbf{W}^{(t)}, \mathbf{G}^{(t)}$

4.2. **Convergence and Classification Theory.**

4.2.1. *Convergence Theory.*

Theorem 4.1 (Convergence of Convex NMF (Ding et al. [2]))**.** *Let $\mathbf{X} \in \mathbb{R}^{m\times n}$ be a real-valued data matrix, and let $\mathbf{G} \in \mathbb{R}_+^{n\times k}$ be a nonnegative factor matrix. Fix $\mathbf{G}$, and apply the update rule for $\mathbf{W} \in \mathbb{R}_+^{n\times k}$, with each column of $\mathbf{W}$ summing to one, as defined in Algorithm 2. Then:*

(1) *The residual $\|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{G}^\top\|_F^2$ is nonincreasing across iterations.*
(2) *The sequence $\{\mathbf{W}^{(t)}\}$ converges to a Karush–Kuhn–Tucker (KKT) fixed point.*

Since the matrix $\mathbf{Z}$ in both Convex SSNMF and Kernel SSNMF takes the form

$$\mathbf{Z}^\top\mathbf{Z} = \phi(\mathbf{X})^\top\phi(\mathbf{X}) + \lambda\mathbf{Y}^\top\mathbf{Y},$$

the update rules for these models reduce to the Convex NMF setting.

Corollary 4.1.1 (Convergence of Convex SSNMF)**.** *Let $\mathbf{X} \in \mathbb{R}^{m\times n}$ be a real-valued data matrix and $\mathbf{Y} \in \mathbb{R}_+^{c\times n}$ be a nonnegative label matrix. Using updates derived from Algorithm 5, with*

$$\mathbf{Z} = \begin{bmatrix} \mathbf{X} \\ \sqrt{\lambda}\,\mathbf{Y} \end{bmatrix},$$

Convex SSNMF exhibits nonincreasing residual and converges to a KKT fixed point.

Corollary 4.1.2 (Convergence of Kernel SSNMF)**.** *Let $\phi(\mathbf{X}) \in \mathbb{R}^{d\times n}$ be an implicit feature map of the data, and $\mathbf{Y} \in \mathbb{R}_+^{c\times n}$ a nonnegative label matrix. Using updates derived from Algorithm 5, with*

$$\mathbf{Z} = \begin{bmatrix} \phi(\mathbf{X}) \\ \sqrt{\lambda}\,\mathbf{Y} \end{bmatrix},$$

Kernel SSNMF exhibits nonincreasing reconstruction loss and converges to a KKT fixed point.

4.2.2. *Classification Using NMF.*

Theorem 4.2. *Given $\mathbf{A} = \mathbf{X}_{train}\mathbf{W}$, the test representation matrix is computed as*

$$\mathbf{G}_{test} = \mathbf{A}^\dagger\mathbf{X}_{test},$$

where $\mathbf{A}^\dagger$ is the Moore–Penrose pseudoinverse:

$$\mathbf{A}^\dagger = \begin{cases} \mathbf{W}^\dagger\mathbf{X}_{train}^T\left(\mathbf{X}_{train}\mathbf{X}_{train}^T\right)^{-1} & \text{if } \mathbf{X}_{train} \text{ is wide,} \\ \mathbf{W}^\dagger\left(\mathbf{X}_{train}^T\mathbf{X}_{train}\right)^{-1}\mathbf{X}_{train}^T & \text{if } \mathbf{X}_{train} \text{ is tall.} \end{cases}$$

Proof. Using $\mathbf{A} = \mathbf{X}_{\text{train}}\mathbf{W}$, we obtain $\mathbf{A}^\dagger = \mathbf{W}^\dagger\mathbf{X}_{\text{train}}^\dagger$. Substituting the standard formulas for the pseudoinverse of a tall or wide matrix yields the result. □

Theorem 4.3. *Given $\mathbf{X}_{train} = \mathbf{A}\mathbf{G}_{train}^T$, the test representation matrix can also be expressed as*

$$\mathbf{G}_{test} = \mathbf{A}^\dagger\mathbf{X}_{test},$$

with

$$\mathbf{A}^\dagger = \begin{cases} \mathbf{G}_{train}^T\mathbf{X}_{train}^T\left(\mathbf{X}_{train}\mathbf{X}_{train}^T\right)^{-1} & \text{if } \mathbf{X}_{train} \text{ is wide,} \\ \mathbf{G}_{train}^T\left(\mathbf{X}_{train}^T\mathbf{X}_{train}\right)^{-1}\mathbf{X}_{train}^T & \text{if } \mathbf{X}_{train} \text{ is tall.} \end{cases}$$

Proof. Given $\mathbf{X}_{\text{train}} = \mathbf{A}\mathbf{G}_{\text{train}}^T$, we compute $\mathbf{A}^\dagger = \mathbf{G}_{\text{train}}^T\mathbf{X}_{\text{train}}^\dagger$. Substituting the pseudoinverse of $\mathbf{X}_{\text{train}}$ yields the expression above. □

Theorem 4.4. *In the kernel setting, with $\mathbf{A} = \phi(\mathbf{X}_{train})\mathbf{W}$, the test representation is given by*

$$\mathbf{G}_{test} = \mathbf{A}^\dagger\phi(\mathbf{X}_{test}),$$

where

$$\mathbf{A}^\dagger = \begin{cases} \mathbf{W}^\dagger\phi(\mathbf{X}_{train})^T\left(\phi(\mathbf{X}_{train})\phi(\mathbf{X}_{train})^T\right)^{-1} & \text{if } \phi(\mathbf{X}_{train}) \text{ is wide,} \\ \mathbf{W}^\dagger\left(\phi(\mathbf{X}_{train})^T\phi(\mathbf{X}_{train})\right)^{-1}\phi(\mathbf{X}_{train})^T & \text{if } \phi(\mathbf{X}_{train}) \text{ is tall.} \end{cases}$$

Proof. Using $\mathbf{A} = \phi(\mathbf{X}_{\text{train}})\mathbf{W}$, we obtain $\mathbf{A}^\dagger = \mathbf{W}^\dagger\phi(\mathbf{X}_{\text{train}})^\dagger$, and substitute accordingly. □

Theorem 4.5. *Alternatively, if* $\phi(\mathbf{X}_{train}) = \mathbf{A}\mathbf{G}_{train}^T$, *then*

$$\mathbf{G}_{test} = \mathbf{A}^\dagger \phi(\mathbf{X}_{test}),$$

where

$$\mathbf{A}^\dagger = \begin{cases} \mathbf{G}_{train}^T \phi(\mathbf{X}_{train})^T \left(\phi(\mathbf{X}_{train})\phi(\mathbf{X}_{train})^T\right)^{-1} & \text{if } \phi(\mathbf{X}_{train}) \text{ is wide,} \\ \mathbf{G}_{train}^T \left(\phi(\mathbf{X}_{train})^T \phi(\mathbf{X}_{train})\right)^{-1} \phi(\mathbf{X}_{train})^T & \text{if } \phi(\mathbf{X}_{train}) \text{ is tall.} \end{cases}$$

Proof. Follows by computing $\mathbf{A}^\dagger = \mathbf{G}_{\text{train}}^T \phi(\mathbf{X}_{\text{train}})^\dagger$ and substituting the appropriate pseudoinverse form. □

Remark. *Observing the different formulae for a tall and wide matrix, one might be concerned about the need to compute the feature map,* ϕ *explicitly. We provide an approximation method to make the algorithm practically usable in all cases. While* $A = KW$ *and* $\phi(X)G^\top$ *are not equal in value, they are functionally similar. Both yield comparable low-rank approximations of the kernel matrix* K *when multiplied by* $G^\top$*. This enables one to use* $A \coloneqq KW$ *as a computational proxy for the feature-space object* $\phi(X)G^\top$*, even though the explicit feature map* ϕ *is never computed.*

Remark. *We observed empirically that the approximation used for reconstructing* $\mathbf{G}_{test}$ *via* $\mathbf{G}_{test} = \mathbf{K}_{test}\mathbf{W}$ *does not significantly affect classification accuracy across multiple datasets. However, if higher precision is desired, particularly in scenarios involving tall feature matrices, the exact inference formula presented in Theorem 4.5 can be used instead. In our experiments, the feature representations (e.g., from embeddings) naturally yield tall matrices. If needed, one can apply a transpose to the input in order to obtain a tall matrix suitable for the exact formulation.*

5. Experiments and Results

In this section, we present numerical experiments for Convex SSNMF (Algorithm 5) and Kernel SSNMF (Algorithm 6). Section 5.1 evaluates both methods on synthetic datasets and standard benchmarks from the UCI Machine Learning Repository. Sections 5.2 and 5.3 apply our algorithms to a curated legal dataset, demonstrating performance comparable to that of SVM classifiers using default settings. Additionally, we show that Kernel SSNMF improves geometric interpretability of the learned representations, as illustrated through non-linear visualizations using t-SNE against Convex NMF.

The code used for the experiments in Section 5.1 is available at `https://github.com/anssinghHarveyMudd/AI_Justice`. Due to the sensitive nature of the legal data, we do not release code or embeddings associated with those experiments, as they may contain identifiable information.

All experiments were conducted using Python 3.9.7 and NumPy 1.23.5.

5.1. **Synthetic Experiments.** We evaluate our algorithm on three datasets: the *Make Moons* dataset from `scikit-learn`, which consists of 500 samples with 2 features and added Gaussian noise of 0.3; the *Digits* dataset, comprising 1,797 instances with 64 features (8×8 grayscale pixel values) and 10 classes (digits 0–9); and the *Breast Cancer* dataset, containing 569 instances with 30 features and 2 classes.

We assess our algorithm along two axes: **(1)** classification accuracy—by using each representation to train a linear-kernel Support Vector Machine (SVM) and comparing against a standard SVM baseline—and **(2)** clustering performance, compared against Convex Non-negative Matrix Factorization (Convex NMF).

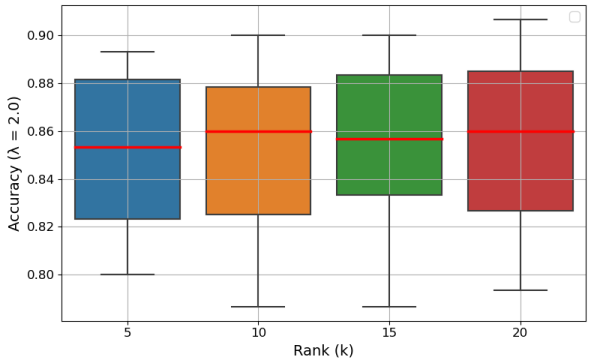
For classification, we tune the hyperparameters to identify the combination of λ and rank that yields the highest accuracy over 10 train-test splits. We additionally evaluate sensitivity to both parameters by varying them individually and visualizing accuracy distributions via boxplots, with medians used to summarize central tendencies.

For clustering, we follow the methodology of Ding et al. [2], which evaluates clustering quality using label permutation via the Hungarian matching algorithm, and reports similarity using Normalized Mutual Information (NMI).

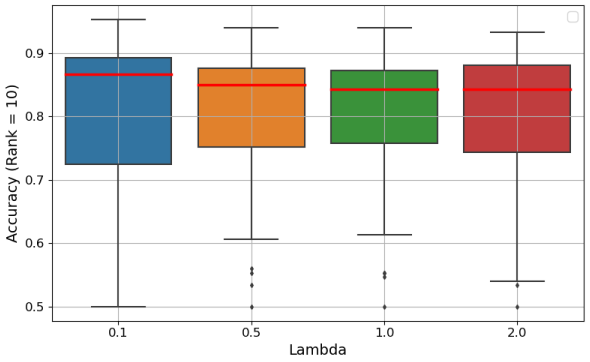
Figures 5, 6, and 7 illustrate classification performance. Despite the geometric constraints imposed by our formulation, the median performance of our method is comparable to that of a default SVM. In some cases, such as Figure 5a, it even marginally outperforms the SVM baseline.

Figure 8 shows clustering accuracy across datasets. Convex SSNMF consistently achieves higher accuracy than Convex NMF, and when the medians are similar, our method exhibits markedly lower variance across trials. This increased stability, driven by the inclusion of label supervision, is a key strength of our approach. We expect this property to generalize to other settings where robustness to initialization is important.

NMI results in Figure 9 echo these trends. On average, Convex SSNMF achieves higher NMI than Convex NMF. Even in cases where the median NMI is slightly lower, our method displays significantly less variability across runs. Given that our method consistently outperforms Convex NMF—and Convex NMF itself outperforms several other algorithms as reported in Ding et al. [2]—we conclude that Convex SSNMF and Kernel SSNMF offer a reliable improvement for both clustering accuracy and consistency.



(A) Convex SSNMF median accuracy for $\lambda = 2.0$ and all presented ranks potentially outperform the mean SVM (linear kernel) accuracy of 85.3%.



(B) Kernel SSNMF (RBF kernel) performs comparably to the mean SVM (RBF kernel) accuracy of 91.3%.

FIGURE 5. Classification accuracy of Convex SSNMF and Kernel SSNMF on the *Make Moons* dataset, under different hyperparameter settings, evaluated over 10 train-test splits. In Figure 5a, our algorithm potentially outperforms SVM.

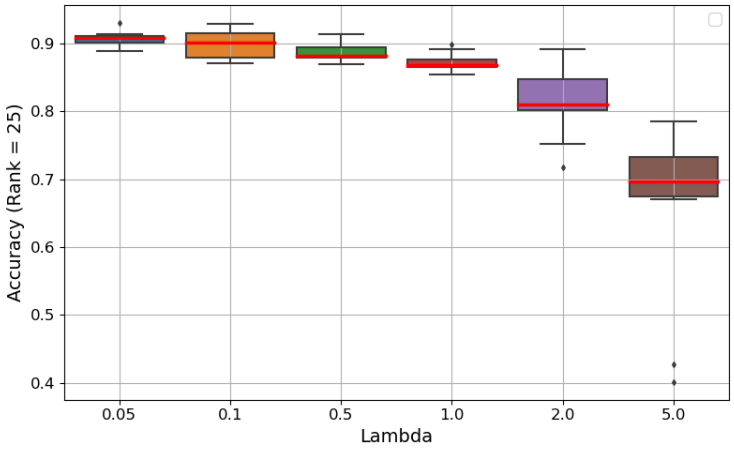


FIGURE 6. Kernel SSNMF (RBF kernel) accuracy on *Digits* for varying λ with $k = 25$, compared to SVM's mean accuracy of 97.52%. Kernel SSNMF shows competitive accuracy under interpretability constraints, with low variance at $\lambda = 0.05$ and $\lambda = 0.1$.

5.2. **Legal Datasets without intentional demographic mislabelling.** In this section, we evaluate our algorithms on the three prompts described in Section 2. For Kernel SSNMF, we construct the kernel matrix using cosine similarity to capture semantic similarity between embedding vectors [3]. Figures 10 and **??** report classification accuracy over 10 train-test splits, comparing Convex SSNMF and Kernel SSNMF with the SVM baseline under linear and nonlinear kernels.

We observe that both SVM and Kernel SSNMF perform better with a cosine kernel compared to a linear one. For the linear kernel, our algorithm consistently performs comparably to—or outperforms—SVM across

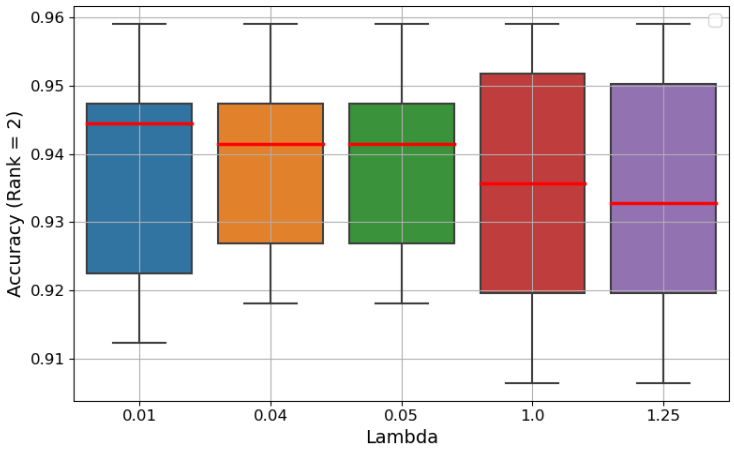
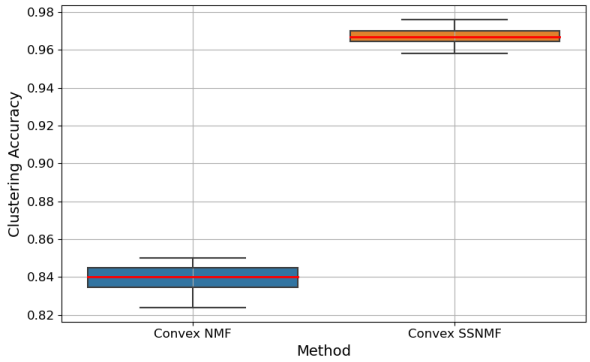
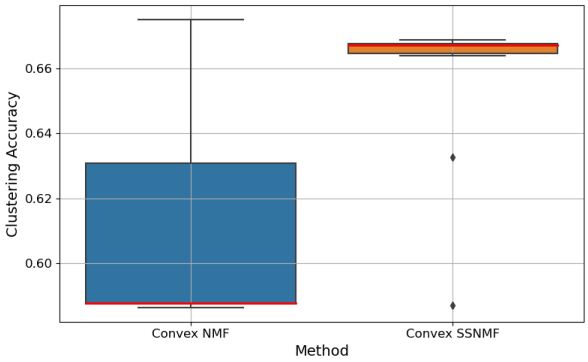


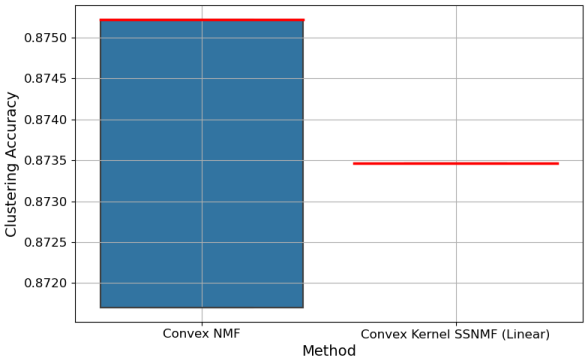
FIGURE 7. Kernel SSNMF (Cubic Polynomial kernel) accuracy on the *Breast Cancer* dataset with fixed rank $k = 2$ and varying λ, compared to the mean SVM accuracy of 97.31%. Median accuracies for lower values of λ (e.g., $\lambda = 0.01$ and $\lambda = 0.04$) approach the SVM baseline closely, with low variance across trials.



(A) *Make Moons*: Convex SSNMF yields significantly higher and more stable clustering accuracy than Convex NMF.

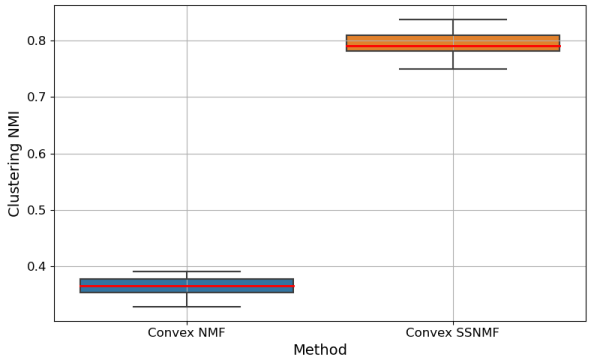


(B) *Digits*: Convex SSNMF exhibits substantially less variance while achieving higher clustering accuracy.

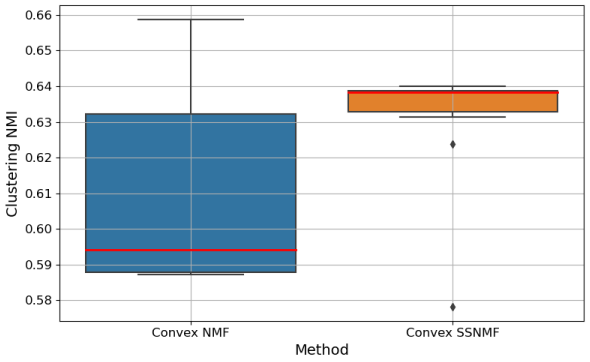


(C) *Breast Cancer*: Despite similar medians, Convex SSNMF is more consistent across runs.

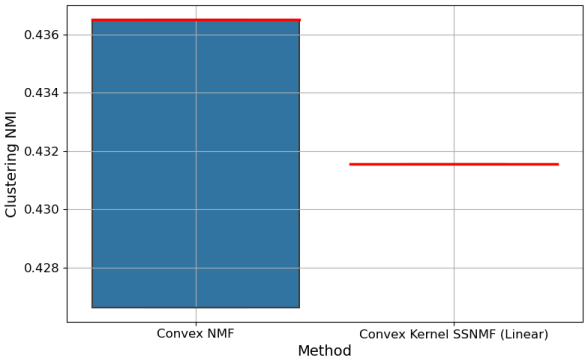
FIGURE 8. Clustering accuracy comparison between Convex NMF and Convex SSNMF across three datasets across 10 trials. Convex SSNMF outperforms Convex NMF on either consistency or accuracy score (or both).



(A) *Make Moons*: Convex SSNMF dramatically improves NMI and reduces variance.



(B) *Digits*: Convex SSNMF yields higher and far more stable NMI.



(C) *Breast Cancer*: Median NMI is similar, but Convex SSNMF is substantially more consistent.

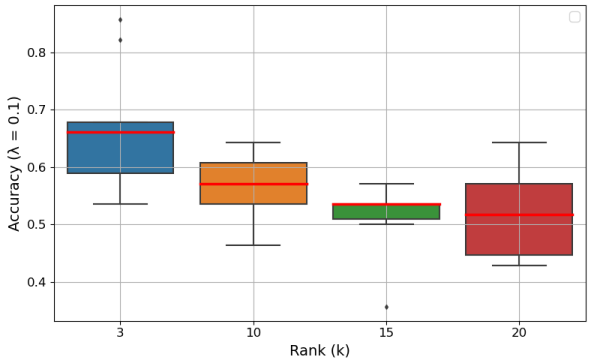
FIGURE 9. Clustering performance (Normalized Mutual Information) across three datasets for 10 train-test splits. Convex SSNMF outperforms Convex NMF on either consistency or higher score (or both).

most prompts, as seen in Figure 10a, while also imposing geometric structure. Most notably, in Figure **??**, Kernel SSNMF outperforms SVM across all prompts with the cosine kernel. This suggests that combining embedding-based similarity along with convex constraints enhances LLM-based classification scores.

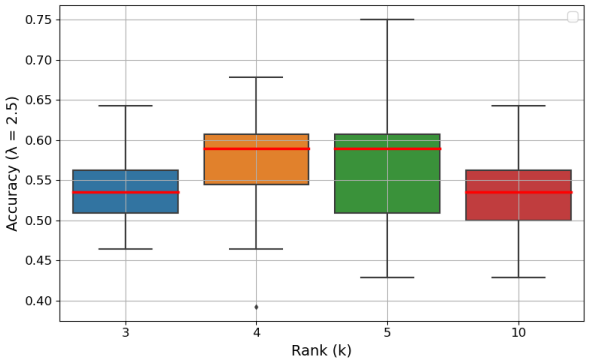
Additionally, in Figure 12, we reinforce the findings presented in the previous section using t-SNE visualizations (perplexity = 30). After applying Kernel SSNMF with convex updates and clustering, we observed clearer class separation in the G_{train} matrix concatenated with the constructed G_{test} matrix from the median accuracy run.

Across all datasets, we observe comparable classification accuracy, with the *Exoneration* and, surprisingly, the *Testimony* prompts yielding the highest performance. The latter is particularly unexpected, as testimony is a subset of evidence, and not all cases include substantive eyewitness accounts. This suggests that more descriptive inputs do not necessarily lead to improved classification accuracy. We had initially expected the *Exoneration* prompt to substantially outperform the others, given that it explicitly asks whether a case should result in exoneration.

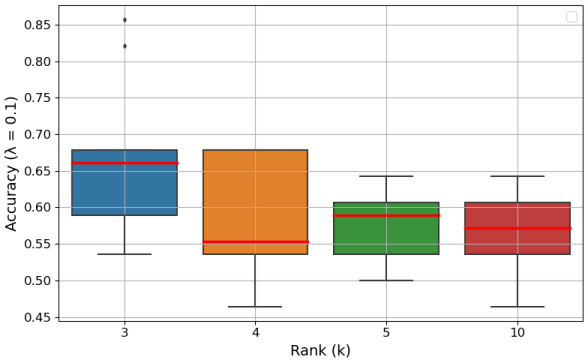
5.3. **Legal Datasets with intentional demographic mislabelling.** In this section, we examine the impact of demographic information on both classification accuracy and the geometric structure of the datasets. To do so, we asked GPT to generate names that it considers demographically neutral. This approach was intentional: since GPT was also used to generate our text embeddings, we aimed to align the anonymization process with the model's own internal representations. We note that on occasion, GPT may introduce minor errors—such as assigning the same name to multiple cases or replacing a formal venue name like "Supreme



(A) Classification accuracy on the Exonerated dataset using Convex SSNMF with rank $k = 3$ and $\lambda = 0.1$. The median accuracy across trials exceeds the SVM baseline of 62.5%, highlighting the method's ability to capture structure even under low-rank and convexity constraints.



(B) Classification accuracy on the Evidence dataset using Convex SSNMF, various ranks and regularization parameter $\lambda = 2.5$ with mean SVM baseline as 61.43%.



(C) Classification accuracy on the Testimony dataset using Convex SSNMF with rank $k = 3$ and $\lambda = 0.1$. Median accuracy exceeds the SVM baseline of 62.5%, indicating that meaningful structure is captured even under low-rank constraints.

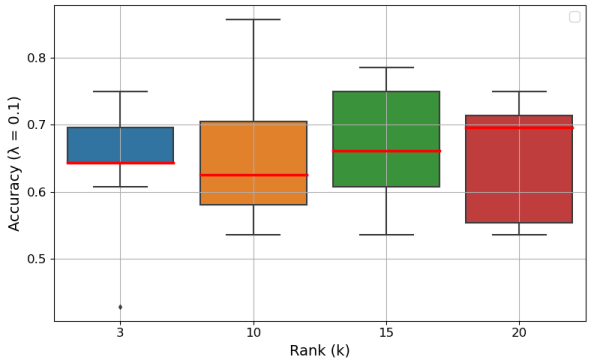
FIGURE 10. Convex SSNMF classification accuracy across 10 train-test splits on various legal embeddings without any demographic mislabelling. Convex SSNMF potentially outperforms the SVM baseline in Figure 10a and Figure 10c.

Judicial Court of Massachusetts" with a placeholder such as "Remy James of Massachusetts." However, such occurrences were extremely rare, and we manually reviewed the affected cases to ensure semantic coherence was maintained. We also instructed GPT to assign as many unique names as possible across cases to reduce the risk of correlating specific names with case outcomes.

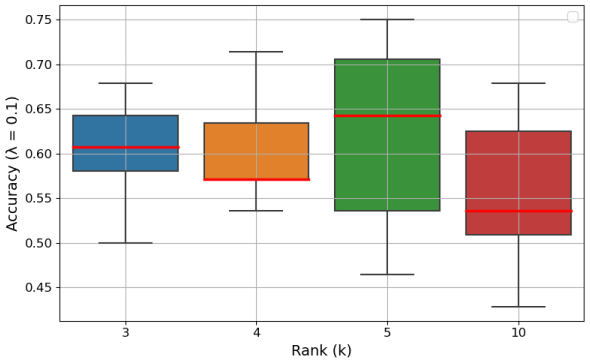
Example of Anonymization and Text Transformation. For example, a previous summary as mentioned in the dataset became (names redacted for privacy reasons):

> *"The case involves two individuals, GKH and JDC, who were convicted of the 1992 murder of RSW based on circumstantial evidence. Both were sentenced to life imprisonment. The Innocence Project is representing the appellants in seeking the release of physical evidence, specifically unidentified hairs found in the victim's hand, for DNA testing that was not available at the time of the trial. The court held that the appellants are entitled to the DNA testing they seek, as it may help prove their innocence."*

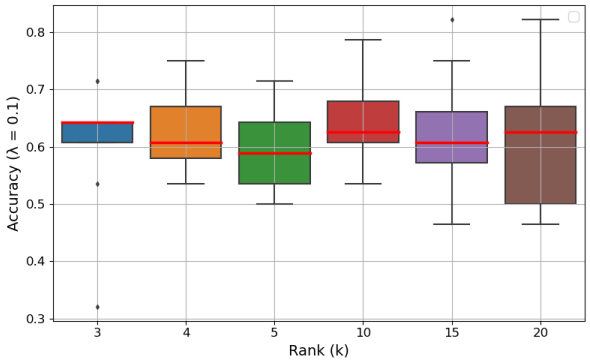
After anonymization and minimal formatting correction, the summary becomes:



(A) Classification accuracy on the Exonerated dataset using Kernel SSNMF (cosine kernel) with rank $k = 20$ and $\lambda = 0.1$. Median accuracy exceeds the SVM baseline of 64.64%, demonstrating strong performance with interpretable structure.



(B) Kernel SSNMF (cosine kernel) accuracy on the Evidence dataset compared to the SVM baseline of 61.79%. Performance improves at rank $k = 5$ while preserving interpretability.



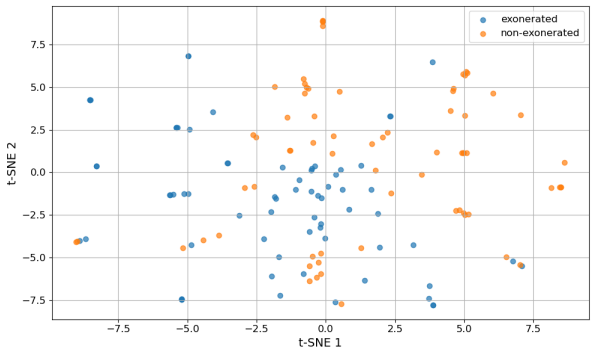
(C) Classification accuracy on the Testimony dataset using Kernel SSNMF with a cosine kernel and $\lambda = 0.1$, across varying ranks. The median accuracy at $k = 10$ performs quite comparably to the SVM baseline of 65%.

FIGURE 11. Kernel SSNMF (cosine kernel) classification accuracy across 10 train-test splits on legal embeddings without demographic mislabelling. The method achieves strong or competitive accuracy across all datasets and may outperform SVM in Figures 11a and 11c, while enforcing geometric constraints in kernel space.

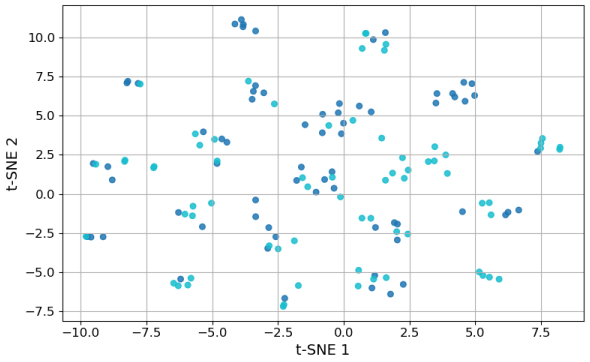
"The case involves two individuals, Rowan Casey and Casey Lee, who were convicted of the 1992 murder of Quinn Riley based on circumstantial evidence. Both were sentenced to life imprisonment. Jules Morgan is representing the appellants in seeking the release of physical evidence, specifically unidentified hairs found in the victim's hand, for DNA testing that was not available at the time of the trial. The court held that the appellants are entitled to the DNA testing they seek, as it may help prove their innocence."

We now conducted all the same experiments as shown in the previous section, using the same ranks and regularization parameters. Across all sets of prompts and kernel choices, we observed an increase in SVM classification accuracy, as shown in Figures 13 and 14. We also noticed greater consistency in the median accuracy scores across these figures. In particular, for Kernel SSNMF with a cosine kernel, there was a notable improvement in median classification accuracy, as illustrated in Figure 14.

Additionally, in the exoneration and testimony prompt settings, our method consistently achieved competitive performance after demographic anonymization. In Figures 14a and 14c, we observed that our method not only potentially outperformed SVM across all datasets, but also substantially improved upon previous median accuracy scores—achieving values such as a 73% accuracy score. Across both linear and cosine



(A) t-SNE of G^T from Kernel SSNMF with a cosine kernel on the Exonerated dataset (rank $k = 20$, $\lambda = 0.1$). The visualization corresponds to the run yielding median classification accuracy (70%) after concatenating the train and test reconstructions.



(B) t-SNE of G^T from Kernel NMF with a cosine kernel on the Exonerated dataset (rank $k = 20$).

FIGURE 12. t-SNE visualizations of the latent representations G^T obtained from Kernel SSNMF and Kernel NMF on the Exonerated dataset without any demographic mislabelling using a cosine kernel. Class separation appears more distinct in the Kernel SSNMF formulation.

kernels, we observed that Testimony and Exonerations, marginally outperformed Evidence prompts which potentially indicate that a more vivid description did not necessarily imply a higher classification accuracy score.

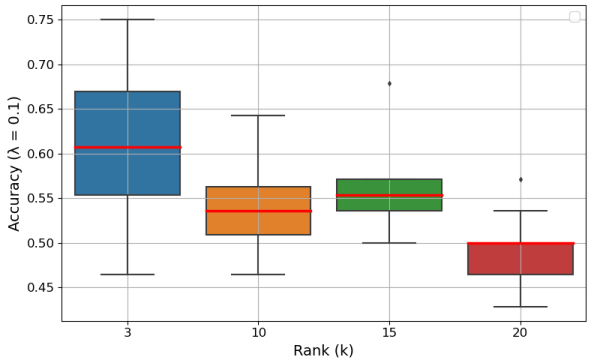
The trend we observed in t-SNE cluster improvements in the previous section is also reflected in our current visualizations. In Figure 15, the clusters appear clearer and more distinct compared to those in Figure 12.

We suspect these consistent gains in classification accuracy stem from three factors: first, demographically biased names may have degraded embedding quality due to biases in the language model's pre-training corpus; second, the presence of common last names (e.g., "Gonzalez") in both the exonerated and non-exonerated groups may have caused the algorithm to conflate features or learn incorrect associations; and third, the relatively small size of our dataset—despite being carefully constructed—places natural limitations on generalization.

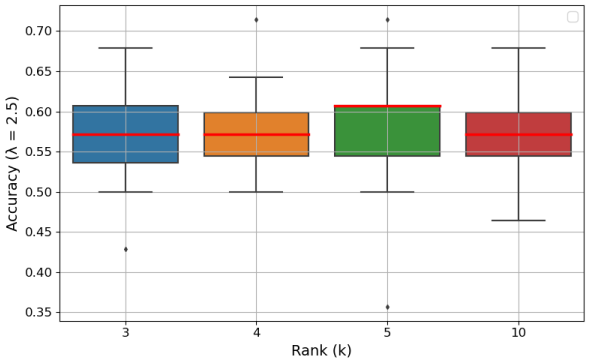
6. Conclusion

In this work, we presented new methods for analyzing exonerations using Large Language Models (LLMs). We introduced two novel algorithms: Convex SSNMF and Kernel SSNMF, along with classification theory and convergence guarantees, which not only achieved potentially comparable classification accuracy to unconstrained methods such as SVM, but also outperformed Kernel NMF on clustering tasks. We emphasized model interpretability—even at the expense of raw accuracy—by imposing convex geometric constraints. Additionally, we proposed two new methods for carefully constructing datasets: first, by creating layered summaries based on various aspects of each case, and second, by attempting to debias AI with AI—asking the model to suggest what it considered demographically neutral names, and then updating the original summaries accordingly for comparison. Finally, we observed that using cosine similarity as a kernel significantly improved classification accuracy, which aligns with our expectations given how the embeddings are constructed.

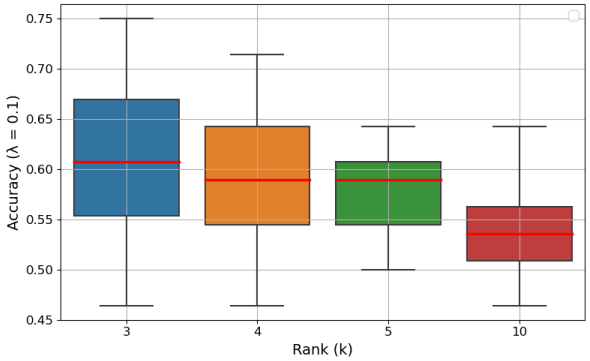
In future work, one might try to utilize a legal LLM to create specialized summaries, as opposed to generalized GPT-4, and experiment with different classes of embeddings such as Legal BERT [4], which are specifically trained on legal texts. One can also try to build legal datasets for cases other than murder and compare whether our debiasing methodology is useful and yields higher classification accuracy in one context versus another. Another idea might be to reconstruct the data matrix and perform text inversions using the method of Morris et al. [8] to semantically interpret topics after tuning pre-trained embedding weights to the



(A) Classification accuracy on the Exonerated dataset using Convex SSNMF. Compared to Figure 10a, anonymization leads to a decrease of approximately 1%–2% at ranks $k = 3$ and $k = 20$, while performance marginally increases or remains stable at $k = 15$ and $k = 10$. The mean SVM accuracy increases from 62.5% to 66.79%.



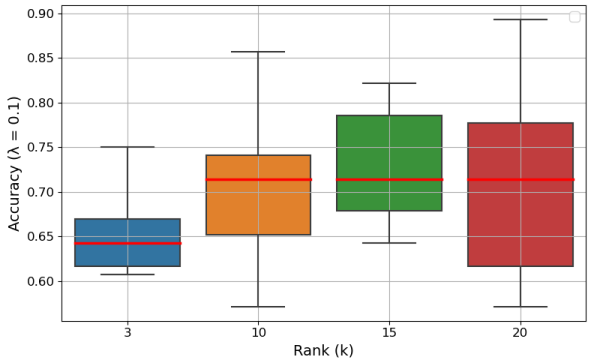
(B) Convex SSNMF accuracy on the Evidence dataset across ranks with $\lambda = 2.5$. Compared to Figure 10b, outlier and median accuracies improve, exceeding 60% at $k = 5$ and stabilizing above 55% across ranks. The SVM baseline also rises from 61.79% to 66.79%.



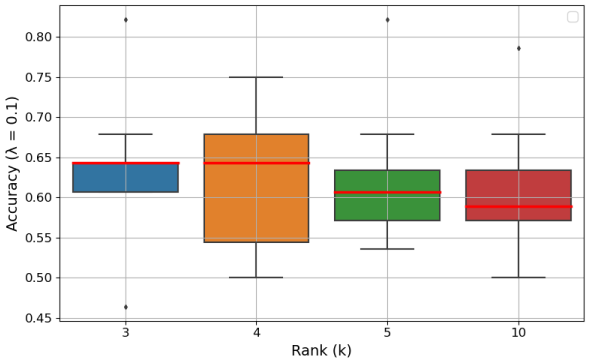
(C) Classification accuracy on the Testimony dataset using Convex SSNMF. Compared to Figure 10c, median accuracies hover around 60% across most ranks. The mean SVM classification accuracy increases from 62.5% to 66.79%.

FIGURE 13. Convex SSNMF classification accuracy across 10 train-test splits on various legal embeddings after demographic anonymization.

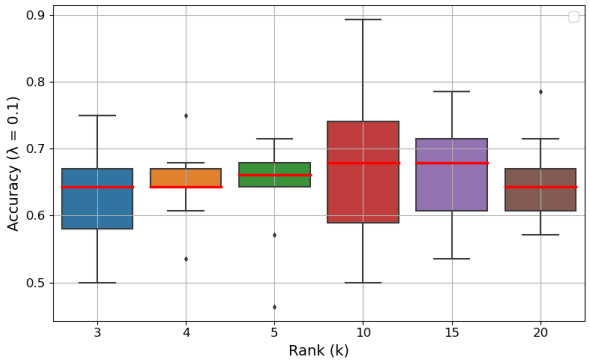
dataset—while being cautious of the fact that tuning might introduce unwanted biases into the model. The application of LLMs to criminal justice cases is one which requires a conscientious effort to ensure debiasing and prioritize model interpretability. Perhaps most surprisingly in this research, AI was able to classify cases which even attorneys would face difficulty with summarizing given document length constraints, indicating the need for greater model transparency and interpretation behind how LLMs actually work.



(A) Classification accuracy on the *Exonerated* dataset using Kernel SSNMF with a cosine kernel. Median accuracy improves, with scores consistently above 54% and outliers reaching 90%. This run potentially outperforms the SVM baseline (mean: 71.07%).

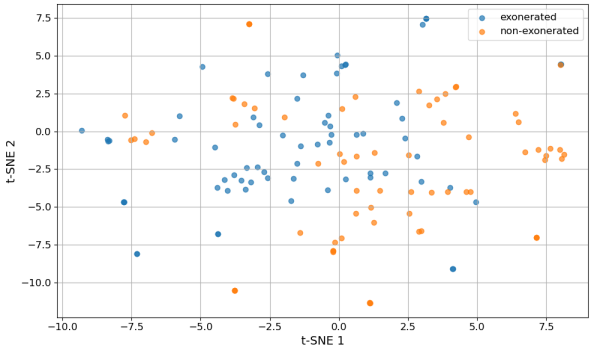


(B) Classification accuracy on the *Evidence* dataset using Kernel SSNMF with a cosine kernel. Median accuracy improves to a stable range of 60%–65%, up from 54%–65%. The SVM baseline also rises to 66.43%, with our method performing comparably.

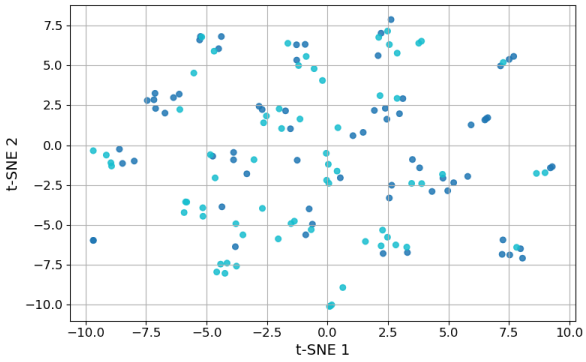


(C) Classification accuracy on the *Testimony* dataset using Kernel SSNMF with a cosine kernel. Median accuracy improved to around 65%–68%, up from 60%, with outliers consistently above 50% and one reaching 90%. Performance potentially exceeds the SVM baseline (mean: 66.7%).

FIGURE 14. Classification accuracy of Kernel SSNMF (cosine kernel) across 10 train-test splits on legal embeddings after demographic anonymization. Our method consistently achieves competitive accuracy and potentially outperforms SVM in Figures 14a and 14c.



(A) t-SNE of $G^\top$ from Kernel SSNMF (cosine kernel) on the *Exonerated* dataset with $k = 20$, $\lambda = 0.1$. Visualization corresponds to the median-accuracy run (73%), using concatenated train and test reconstructions.



(B) t-SNE of G^T from Kernel NMF with a cosine kernel on the Exonerated dataset (rank $k = 20$).

FIGURE 15. t-SNE of $G^\top$ from Kernel SSNMF and Kernel NMF on the *Exonerated* dataset with demographic mislabelling (cosine kernel). Kernel SSNMF shows clearer class separation than Kernel NMF and may even outperform the pre-anonymization visualization in Figure 12(A).

Acknowledgements

The authors gratefully acknowledge Mr. Mike Semanchik of the Innocence Center and Professor Marissa Bluestine of the University of Pennsylvania's Quattrone Center for their valuable guidance during this project. We also thank the UCLA Computational and Applied Mathematics REU program, Harvey Mudd College, and the National Science Foundation (grant DMS-2011140) for their support.

References

[1] Sungjae Cho. Lee and seung (2000)'s algorithms for non-negative matrix factorization: A supplementary proof guide, 2025.
[2] Chris H.Q. Ding, Tao Li, and Michael I. Jordan. Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1):45–55, 2010.
[3] Yeli Feng. Semantic textual similarity analysis of clinical text in the era of llm. In *2024 IEEE Conference on Artificial Intelligence (CAI)*, pages 1284–1289. IEEE, 2024.
[4] Kai Kugler, Simon Münker, Johannes Höhmann, and Achim Rettinger. Invbert: Reconstructing text from contextualized word embeddings by inverting the bert pipeline. *arXiv preprint arXiv:2109.10104*, 2021.
[5] Daniel Lee and H Sebastian Seung. Algorithms for non-negative matrix factorization. *Advances in neural information processing systems*, 13, 2000.
[6] Daniel D Lee and H Sebastian Seung. Learning the parts of objects by non-negative matrix factorization. *nature*, 401(6755):788–791, 1999.
[7] Hyekyoung Lee, Jiho Yoo, and Seungjin Choi. Semi-supervised nonnegative matrix factorization. *IEEE Signal Processing Letters*, 17(1):4–7, 2009.
[8] John X Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M Rush. Text embeddings reveal (almost) as much as text. *arXiv preprint arXiv:2310.06816*, 2023.
[9] National Registry of Exonerations. The national registry of exonerations, 2024. Accessed: 2024-06-26.
[10] OpenAI. New embedding models and api updates, 2024. Accessed: 2025-05-23.